

ROS2 Midterm Exam -- Section E Report

010243407 Robot Operating System Programming -- Production and Robotics Engineering, KMUTNB

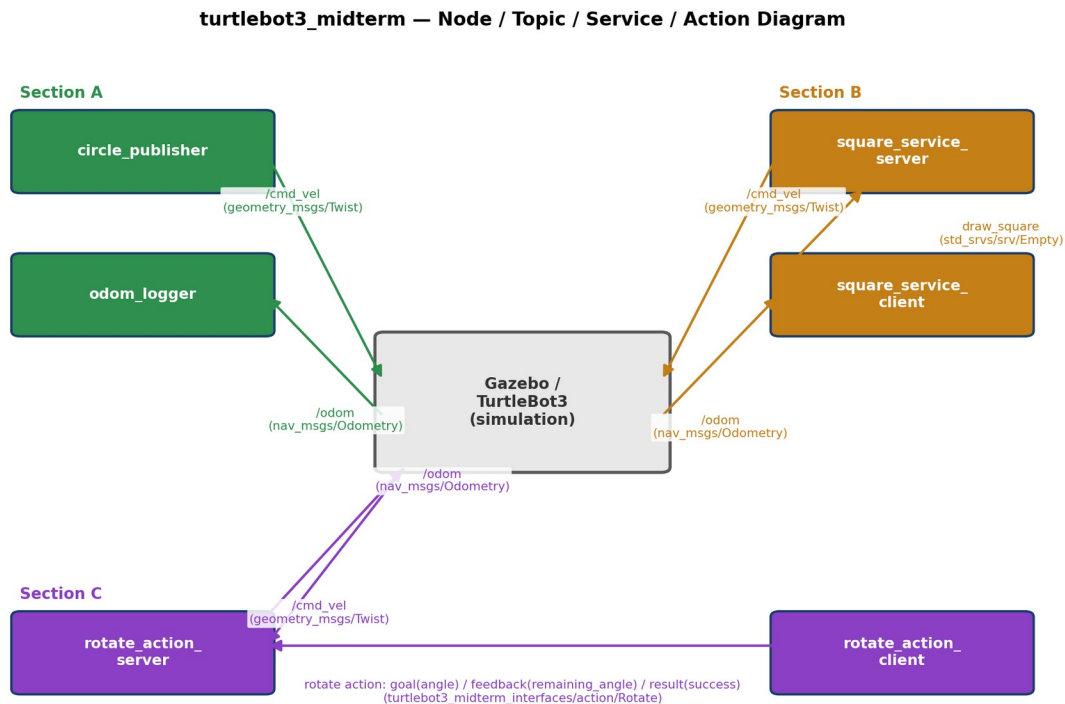
Group members

- Phokchaman Phonchuai -- Student ID 6701023610127 -- S6701023610127@kmutnb.ac.th
- Ratchadapa Yooyatmark -- Student ID 6701023611085 -- s6701023611085@kmutnb.ac.th

ROS2 packages: turtlebot3_midterm (holds the nodes) and turtlebot3_midterm_interfaces (holds our custom Rotate.action).

1. Node diagram

The picture below shows all six nodes we wrote, grouped by exam section. It also shows the topics, the service, and the action that connect them to Gazebo/TurtleBot3 and to each other, plus the message type on every connection.



2. Screenshots -- each section working

Section A -- circle_publisher drives the robot in a circle. At the same time, odom_logger keeps printing x, y, and yaw:

3. The Kp value we used, and how we found it

The P controller inside `rotate_action_server.py` works out $\text{cmd_vel} = K_p \times \text{error}$, and this value is capped at $\text{max_ang_vel} = 1.0 \text{ rad/s}$. We tried four Kp values (0.5, 1.0, 1.5, 2.0) in Gazebo. Each time we sent the same 180-degree (3.14159 rad) goal and reset the Gazebo world first, so every test started the same way and the results could be compared fairly:

Kp	Time to finish (s)	Overshoot?	What we saw
0.5	5.62	No	The fastest of the four. Smooth the whole time, no shaking.
1.0 (the starting value)	7.19	No	Slower than we expected. The speed command hit the $\text{max_ang_vel}=1.0 \text{ rad/s}$ limit while the error was still big.
1.5	7.01	No	Almost the same as $K_p=1.0$. Still hit the speed limit for most of the turn.
2.0	7.40	No	The slowest of the four. A bigger Kp did not help once the speed limit was hit.

Our final choice: $K_p = 0.5$.

This was the fastest of the four values we tried (5.62 s, compared with 7.0-7.4 s for the rest), and it never overshoot the target. At first this felt strange, since a bigger Kp is usually expected to move faster. But the log data shows the reason: once the angle left to turn is bigger than about 60 degrees, every Kp of 1.0 or higher asks for a turning speed above the $\text{max_ang_vel} = 1.0 \text{ rad/s}$ limit, so it just gets capped down to the same 1.0 rad/s. Because of this, raising Kp past 1.0 did not add any extra speed for most of the turn. $K_p = 0.5$ stays under that speed cap for longer, so it slows down more smoothly on the way to the goal, and this ends up finishing sooner overall. This tells us that a P controller does not only follow the $K_p \times \text{error}$ formula -- its real behaviour also depends on the robot's speed limit and its inertia in the simulation.

4. Reflection

The hardest part: splitting the three ROS2 communication styles (topic, service, action) into the right packages was the hardest idea to fully understand, especially working out why the custom action needed its own separate `ament_cmake` package. On the coding side, the trickiest bug was a crash when we pressed Ctrl+C in `circle_publisher.py`. This happened because `rclpy` has its own Ctrl+C handler, and it could close the connection before our own cleanup code ran, which caused a "publisher's context is invalid" error. The other tricky bug was in `square_service_server.py`, where the service code called `rclpy.spin_once()` while the node was already being spun by `rclpy.spin()` in `main()` -- spinning inside another spin like this can get stuck. We fixed both: by checking `rclpy.ok()` before every step that needs the connection to still be open, and by changing `square_service_server` to use a `MultiThreadedExecutor` with a `ReentrantCallbackGroup`, so the odometry updates and the service work run on two separate threads instead of one spin sitting inside the other.

What we would do differently next time: tune K_p before recording the demo video, not after. We first recorded with a default $K_p = 1.0$ that we had never actually tested, and only found out while tuning that $K_p = 0.5$ was both safer and faster. We would also add the same clean shutdown handling to every node right from the start, instead of finding and fixing it node by node after seeing error messages in our own recordings.

5. How we used generative AI

We used an AI assistant (Claude) to write a first draft of all six nodes (the topic publisher/subscriber, the service server/client, and the custom action server/client), including the quaternion-to-yaw formula and the basic shape of the P controller. After that, we did the rest ourselves: (1) checked every node against the exam's message types, topic/service/action names, and scoring rules; (2) found and fixed the Ctrl+C bug and the nested-spin bug described above, after we saw them happen in our own test runs; (3) ran the four K_p tests in this report ourselves in Gazebo, and picked $K_p = 0.5$ from our own results, not from a number the AI suggested; and (4) we understand our work.